SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

Department of Artificial Intelligence and Machine Learning

AgentOS: A Case Study in Memory Management and Secure File Systems

Avani Jain, Aparajita Mazumdar, Ishaan Chand, Raghunandana
Department of Artificial Intelligence and Machine Learning, Symbiosis Institute of Technology, Pune Symbiosis International University, Pune, India.

Student name and PRN:

Student Name	PRN
Avani Jain	25070126044
Aparajita Mazumdar	25070126033
Ishaan Chand	25070126076
G.Raghunandana	25070126065

Course name: Operating Systems

Faculty Name: Dr. Sheetal Borhade

Submission Date: 23rd October 2026

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

Department of Artificial Intelligence and Machine Learning

Abstract

As the computing environment becomes increasingly resource intensive the efficient management of memory and files becomes an important part of operating system design. Workspaces where multiple tasks share a lot of resources also give challenges which are related to memory availability, file organization and controlled access to information. Our case study explores these challenges through AgentOS which is a simulated agent oriented workspace which is designed to examine the fundamental operating system mechanisms. Our study considers how different page replacement strategies respond to limited memory and how the structured file management and access permissions support the handling of shared resources. Using controlled scenarios and predefined test cases our project evaluates the behaviour of FIFO and LRU page replacement algorithms alongside essential file system operations. Our observations provide a basis for comparing their behaviour, understanding their practical significance and also identifying the limitations of the simulation. Our study aims to bridge the gap between theoretical operating system concepts and their application in a shared computational environment.

Keywords: Operating Systems, Memory Management, FIFO, LRU, File System, Directory Structure, Access Permissions, Simulation.

1. Introduction

Operating systems plays a very important role in the management of system resources, organizing files and also controlling access to data. When multiple tasks are operated within a shared workspace then the efficient memory management and proper file system coordination becomes important for maintaining reliable operations. Limited memory can result in page faults and page replacement while shared file access requires correct mechanisms to organize, access and protect stored information.

Our case study explores these operating system concepts using a hypothetical workspace which contains three agent related tasks which are the Research Agent, Coding Agent and Report Agent. These tasks actually share a limited memory budget which then interacts with files within a common project directory.

The memory management component here compares the First In First Out and Least Recently Used page replacement algorithms. The file system component then inspects different methods including hierarchical directory organization, file attributes, path resolution, file operations and access permissions. Through small software simulations our project shows how established operating system mechanisms can be evaluated under a defined workload. The main objective is to understand the behaviour of these mechanisms through reproducible experiments and then analyse their results and identify their advantages, limitations and practical significance.

1.1 Contemporary Setting: The Omarchy Case Context

In modern software engineering, developers are increasingly employing autonomous AI coding agents to edit files, execute test suites and coordinate multi-step workflows [8]-[10]. To ground our operating systems study in real-world practices, this investigation examines Omarchy, an Arch-based Linux desktop distribution explicitly tailored for workflows with AI coding agents [1]-[4]:

- Desktop Infrastructure [1]: Combines Arch Linux with the Hyperland Tiling Wayland compositor and Quickshell desktop widgets, creating an agile, scriptable workstation.
- Agent Integration [2]: Features built-in launchers for multiple coding agents, default agent configuration hooks, and an Omarchy skill enabling agents to query desktop commands and window states.
- CLI and User Dotfiles [3][4]: Exposes workspace management through the Omarchy command-line utility and organizes configuration into modular user dotfiles.

While Omarchy demonstrates how developers interact with coding agents in a modern Linux desktop, it operates under standard single-user Linux semantics. It does not introduce specialized kernel memory managers for agents or enforce fine-grained per-role file isolation. This motivates the need for our bounded, principled pedagogical model. [1]-[4]

1.2 Research Question & Two-Layer Methodology

- Core Research Question**:** How do conventional operating systems principles regarding limited physical memory and organized, protected file systems help analyze a hypothetical multi-agent workspace, and what do reproducible simulations demonstrate about those mechanisms?

The evaluation is structured around two complementary layers:

1. Documentary Case Analysis: Extracting observable architectural decisions and resource boundaries from Omarchy's official documentation [1]-[4].
2. Bounded Pedagogical Simulation (AgentOS): Implementing deterministic software simulations modeling three cooperative roles (Research, Coding, and Report) sharing $m = 3$ physical memory frames and a protected file directory hierarchy.

1.3 Claim Boundaries and Negative Declarations

1. Simulation Boundary: AgentOS is a report-first discrete simulator. It does not implement a new OS kernel or run real LLMs or allocate physical RAM pages or even execute live system calls against the host disk.

2. Role Abstraction: The labels Research Agent, Coding Agent and Report Agent designate simulated workload profiles not autonomous cognitive entities or OS processes.

3. Disambiguation: Omarchy is desktop tooling; AIOS [5] is an academic agent OS kernel architecture; MemGPT [6] is prompt context tiering; AgentOS is an educational systems evaluation.

2. Problem Statement and scope

AgentOS workspace is representative of an environment where multiple tasks share limited memory while accessing a common project directory. This creates two major operating system challenges.

The first challenge is memory management. Since the available memory frames can't accommodate every referenced page at the same time, the system must determine which page has to be removed when all frames are occupied and a new page requests a frame. Depending on the page replacement algorithm that is used the page fault will differ according to the page reference sequence that is used.

The second challenge is file system management. Multiple tasks require access to shared files, however, not every task should be authorised to perform all file operations. Invalid file paths, unauthorized access and even improper management of file operations can lead up to causing unintended modifications and also errors. Therefore the system requires a structured directory hierarchy, valid path resolution, supported file operations and proper access control rules.

The project addresses these challenges by using a simulated workspace. FIFO and LRU are evaluated using identical page reference sequences, while the file system component demonstrates directory management, file operations, as well as permission enforcement.

The scope of AgentOS is limited to an educational simulation of operating system concepts, it does not involve developing a new operating system kernel or modifying actual user files.

Integration Perspective: Four Systems Challenges

1. Paging Divergence: Two replacement policies receiving identical reference strings diverge in frame residency upon a single recency refresh, leading to completely separate downstream eviction paths.
2. Allocation Anomalies (Belady's Anomaly)**:** Expanding physical frame allocation ($m \rightarrow m + 1$) is conventionally assumed to monotonically reduce or preserve page fault rates. We visually proved that FIFO violates this assumption on specific workloads whereas LRU guarantees monotonic improvement.
3. Protection Boundaries & Information Leakage**:** In multi tenant workspaces authorization must be validated before the node existence resolution to prevent any unauthorized agents from probing directory contents.
4. Lost-Update Prevention: When agents simultaneously read, compute and write shared deliverable, optimistic concurrency control must detect stale version overwrites.

3. Memory Management in the AgentOS Context

AgentOS as we call it, is Our Group's Prototype, which has been developed for studying one of the most important aspects of an Agent-Oriented Workspace, The Resource Management Mechanisms, particularly the Memory Management Subsystem. So, in an Agent-Oriented System, where Tasks may repeatedly request Information while operating under a Bounded Resource Budget, proper Resource Management becomes particularly important. AIOS proposes OS-like Services for LLM-Based Agents, including Memory and Storage Management, while MemGPT, explores Context Management inspired by Hierarchical Memory Systems [5][6] . These Systems provide the Research Background for Our Study, however, neither AIOS nor MemGPT is actually implemented within this particular Module.

The Memory Management Component of AgentOS, focuses on one specific question, when only a Limited Number of Page Slots are available, which previously loaded Page should be replaced when a New Page is requested? To study this, Our Implementation compares two main Page Replacement Algorithms, First-In, First-Out (FIFO) and Least Recently Used (LRU), using the same Fixed Reference Sequence. Each Request, produces an observable Page Hit or Page Fault, while every Replacement Operation records the Page that has been Evicted. So, by keeping the Reference Sequence fixed, We can reproduce the same behaviour and directly compare how these 2 Replacement Policies, perform under identical Conditions.

However, it is important to clarify one thing, AgentOS is a Bounded Simulation, and not a Complete Operating System or a Production-Ready LLM Context Manager. This Module does not allocate Actual Physical RAM, create Operating-System Processes, run Autonomous Agents, or reproduce the Internal Mechanisms of AIOS, MemGPT or Omarchy. Instead, the broader AgentOS Case Study, uses this Subsystem to demonstrate one concrete Resource Management Concept, and how different Policies can produce different outcomes under a Limited Resource Budget.

3.1 Demand Paging, Stack Algorithms, and Belady's Anomaly

In Demand-Paged Virtual Memory, the Logical Address Space is divided into Equal-Sized Pages, while the Physical Memory is divided into Frames [5].

So, whenever a process references a Page which is not currently present in Physical Memory, a Page Fault Trap is generated. This is one of the fundamental concepts behind Virtual Memory, because the Operating System must determine which Pages should remain in Memory and which ones need to be brought in or replaced.

- **Page Hit vs. TLB Hit:** A Translation Lookaside Buffer (TLB), is basically a Hardware Cache that stores Virtual-to-Physical Address Mappings, allowing Address Translation to take place more efficiently when a TLB Hit occurs. However, in AgentOS, We are measuring something different, Page Residency. Here, a Page Hit simply means that the requested Abstract Page Identifier, is already present in one of the 3 Simulated Physical Frames. So, a Page Hit in Our Simulation, should not be confused with a Hardware TLB Hit.
- **Stack Algorithms:** Now, an Algorithm is classified as a Stack Algorithm, when the Set of Pages resident in an (m)-Frame Configuration, remains a strict subset of the Pages resident in an ((m+1))-Frame Configuration at the same time (t). In other words, increasing the Number of Frames, should not cause the Algorithm to lose a Page which was already resident under the smaller Configuration. This Property, is represented as:

$$M(m,t) \subseteq M(m+1,t)$$

- **Least Recently Used (LRU):** The LRU Algorithm, operates on one main principle, The Page which has not been Used for the Longest Time, is selected for Replacement. Because LRU retains the (m) most recently referenced unique Pages in an (m)-Frame Configuration, increasing the Capacity to (m+1) allows the additional, next-most-recently referenced Page to be retained without removing the Pages already present in the smaller Configuration. Therefore, LRU satisfies the Stack Inclusion Property, and is mathematically immune to Belady's Anomaly. This means that increasing the Number of Frames, cannot increase the Number of Page Faults:

$$\text{Faults}(m+1) \leq \text{Faults}(m)$$

- **First-In, First-Out (FIFO):** The FIFO Algorithm, follows a different approach, The Page which was Loaded into Memory First, is the one selected for Replacement, regardless of whether it was recently used or not. So, unlike LRU, FIFO does not maintain the Stack Inclusion Property. This can lead to a particularly interesting Problem, where increasing the Number of Available Frames, can actually increase the Number of Page Faults. This behaviour is known as Belady's Anomaly, and is represented as:

$$\text{Faults}(m+1) > \text{Faults}(m)$$

Therefore, the difference between FIFO and LRU, is not merely about which Page gets replaced, but also about the Mathematical Properties of the Replacement Algorithm itself.

3.2 Hierarchical File Systems & Longest-Prefix Access Control

In a Hierarchical File System, Directories are organised in a Tree Structure, beginning from the Root Directory, represented as /. Now, when multiple Agents access files and directories within an Agent-Oriented Workspace, the System must determine which Operations each Agent is actually permitted to perform. For this, Access Permissions can be mapped between Agents, acting as Subjects, and Path Prefixes, acting as Objects, across 4 main Privileges, ({r,w,c,d}), representing Read/List, Write/Overwrite, Create/mkdir and Delete.

- **Longest-Prefix Matching:** When an Agent (S), requests an Operation (O) on a Path (P), the System determines the Effective Permission by locating the Longest Matching Rule Prefix. In simple terms, a more specific Rule defined for a Child Directory, takes precedence over a broader Rule inherited from its Parent Directory. So, instead of treating every Path as if it had the same Permissions, the System evaluates which Rule most specifically applies to the Requested Path.
- **Security Check Before Existence:** One of the most important Security Principles here, is that Authorization must be checked before revealing whether a Path Exists. This prevents Unauthorized Agents from probing Directory Contents or discovering the Structure of the Workspace through differences in the System's Responses. Therefore, the Authorization Check, is not simply an additional Validation Step, but a mechanism intended to prevent Unauthorised Information Disclosure.
- **Optimistic Concurrency Control:** In AgentOS, each File maintains a Monotonic Version Counter, which is incremented after every Successful Write Operation. So, when an Agent attempts to modify a File using an expected_version that does not match the Current Version, the System immediately returns a CONFLICT Status. This is particularly important, because multiple Agents may attempt to modify the same Resource, and without Version Checking, one Agent could unintentionally overwrite Changes made by another. By checking the Expected Version before accepting a Write, the System can detect such Conflicts and prevent updates based on Stale File Versions.

3.3 Literature Review Matrix

Comparative Dimension	Omarchy Manual [1]-[4]	AIOS [5]	MemGPT [6]	AgentOS
System Category	Linux Desktop Distribution	Research Agent Operating System	Virtual Context Management	Pedagogical OS Case Study
Implementation Layer	Arch Linux, Hyperland, Quickshell	Intermediate kernel between LLM & OS	Prompt context software library	Python deterministic discrete simulator
Memory Management	Linux kernel virtual memory, cgroups	Agent context scheduling & memory	Memory tiers (main vs. archival DB)	Demand paging (3 frames, FIFO vs. LRU)
File System / ACL	Standard POSIX permissions (ext4/Btrfs)	Tool cache & agent storage manager	Vector database document storage	In-memory tree with longest-prefix ACLs
Multi-Tenancy	Single user running agent launchers	Multi-agent process table & scheduler	Multi-session chat context	3 collaborative roles (Research, Coding, Report)
Evaluation Focus	Developer desktop usability	Token throughput & waiting time	Recall accuracy & multi-session depth	Verified hit ratios, Belady curve, ACL audit

4. Case Description

4.1 Table 1: Omarchy Source-Based Feature and Claim Ledger

Manual Chapter	Documented Architectural Feature	Observable Design Choice	System Inference & AgentOS Boundary
Welcome [1]	Arch Linux base with Hyperland and Quickshell	Lightweight tiling compositor with scriptable IPC	Provides an efficient GUI desktop; does not alter Linux kernel resource arbitration.
AI [2]	Agent launchers, default agent selection, Omarchy skill	Desktop integration for multiple coding agents	Agents execute within the user's host environment; fine-grained per-role file access is not enforced.
CLI [3]	`omarchy` command-line management interface	Unified terminal entry point for desktop tools	Simplifies tool invocation; does not provide kernel-level process scheduling.
Dotfiles [4]	Modular, user-editable dotfiles in Git repo	User-configurable system behavior	Dotfiles define workspace layout; concurrent agent write conflicts require external version control.

Table 2: Benchmark Reference Workloads

Workload	Reference String	Length	Locality Profile & Theoretical Purpose
Workload A	`[7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1]`	22	Unit 4 Notes string; high temporal locality on {0, 2, 3}.
Workload B	`[1, 2, 3, 1, 4, 2, 5, 1, 2, 3]`	10	Mixed locality with rapid working set shift.
Workload C	`[1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4]`	16	Cyclic loop with working set size W = 4 > m = 3 (pure thrashing).
Workload D	`[1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]`	12	Classic Belady reference string evaluated across 1 to 5 frames.

Table 3: Multi-Agent Access Control Matrix

Path Prefix	Research Agent	Coding Agent	Report Agent	Rule Resolution Precedence
`/workspace`	`r`	`r`	`r`	Base inherited read permission
`/workspace/notes`	`rwcd`	`r`	`r`	Specific override (Research sandbox)
`/workspace/results`	`r` (inherited)	`rwcd`	`r`	Specific override (Coding sandbox)
`/workspace/reports`	`r` (inherited)	*Denied*	`rwcd`	Specific override (Report sandbox)

4.2 AgentOS Memory Workload

The illustrative AgentOS workspace uses three workload roles: Research Agent, Coding Agent, and Report Agent. These labels describe task types that may revisit project information. They are not any autonomous agents or separate operating system processes implemented by the memory module. The simulator actually receives anonymous page identifiers and does not know the content, owner or even meaning of a page.

The experiment uses three frames and the following 22-reference sequence for both policies:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

A hit occurs when the requested page is already present in a frame. A fault will occur when it is absent. If a fault occurs while a free frame remains the requested page is loaded into that frame directly. If all three frames are occupied the selected policy chooses a resident page to replace. The page hit ratio is calculated as the number of hits divided by the total number of references.

Both runs begin with empty frames and use identical input, capacity and counter definitions. Page identifiers are abstract values. They do not represent bytes, prompt tokens, LLM context blocks or even measured agent states.Our experiment therefore isolates replacement policy behavior rather than measuring an actual AI workload.

5. Case Description: Incident 001: The Overloaded Agent Workspace

5.1 Overview of the workspace

The hypothetical AgentOS workspace represents an engineering project where the three agent like tasks share a common computation, memory and a common project directory.

The three tasks are:

- Research Agent: This agent reads and revisits source notes required for the project.
- Coding Agent: This agent accesses source notes and updates the file containing experimental results.
- Report Agent: This agent reads the experimental results and thus prepares the final report.

The project directory contains the following files:

- source_notes.txt: this file stores research notes and supporting information.
- results.csv: This file stores experimental results generated by the coding task.
- final_report.md: This file contains the final report that is prepared using the available results.

These files are organized within a simulated directory structure. The defined tasks interact with the files using defined operations, they don't directly make any modifications to actual computer files.

5.2 Description of the incident

Incident 001 happens when all three tasks are working on the same project with limited memory resources. The memory component has a fixed number of frames. When a task requests a page that is not currently loaded into the memory, a page fault occurs. If all memory frames are occupied, the system has to select a page to be evicted before the requested page gets loaded. FIFO and LRU are responsible for removing the pages, FIFO removes the page that had arrived first while LRU removes the page that hasn't been used in the longest time.

Concurrently, the tasks perform file operations on the project directory. Research Agent reads the source notes, Coding Agent updates results.csv, and Report Agent accesses the results in order to prepare final_report.md. These operations require the file system to locate the files then determine whether the operations being requested are supported, and verify whether the requesting task has all the permissions it needs.

Hence, Incident 001 gives a common scenario that can be used for assessing both memory management behaviour and file system operations within the shared workspace.

5.3 Relevance to operating-system concepts

The memory management component of AgentOS is used to simulate paging, page faults, page hits, page replacement, and frame allocation. The comparison of FIFO and LRU emphasises how different replacement policies handle the same memory reference sequence and how it affects the page fault behaviour.

The file system component of AgentOS illustrates the hierarchical directory organization, file attributes, file access, pathnames, read or write operations, as well as access permissions. It shows how the files can be arranged and how different file operations can be permitted or rejected depending on the predefined rules.

These agent like tasks only act as labels for the simulated workload. AgentOS does not actually implement any autonomous agents nor does it manage the internal memory of a language model.

5.4. File-System Management and Coordination

5.4.1 File-system organization

In a file system, files and folders are organised for easy access, in AgentOS files are arranged in folders and subfolders. A directory tree depicts how its arranged and the pathname helps in locating a particular file, which allows the simulated agents to find a file and perform the needed operations.

For instance, in the AgentOS simulator the Research Agent accesses the source_notes.txt file while the Coding Agent updates another file called Results.csv, the Report agent the uses this to create the final_report.md file, the files and their locations are managed within the program itself, i.e., all operations occur within the simulated environment hence making no changes to any actual files on the computer

5.4.2 File attributes and operations

Every file has certain attributes that are used to identify and manage the file in a file system. Depending on how it has been implemented, the attributes may include it name, type, size location or any other metadata that may be relevant. In AgentOS attributes are dependent on the features that are implemented in the simulation.

The AgentOS file system component supports the basic operations that are required to demonstrate file management, including the following operations:

- Create: Creates a simulated file or directory where supported.
- Read: Retrieves the contents or stored information of an accessible file.
- Write: Updates the simulated contents of a file when the operation is permitted.
- Delete: Removes a simulated file or directory when the operation is permitted.
- List: Displays the entries contained within a directory.
- Path resolution: Identifies the target file or directory associated with a supplied pathname.

The operation performed by the file systems will be dependent on the features implemented in the program.

5.4.3 File access and permissions

When multiple tasks are being performed on files in the same directory the agent may attempt to perform prohibited file operations on shared files. To avoid this from happening access permissions control which tasks have authorisation to perform certain operations on particular files.

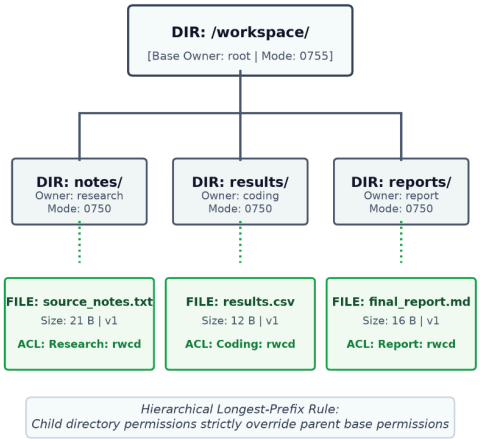
In AgentOS every task is assigned certain operations that it can perform. For instance one task may have the ability to read or modify a file but not delete it while another task may have the permission to delete a file but may not be permitted to read or modify the file.

When an agent requests a file operation, the file system check the file path and the permissions the agent has. If the request is valid and is permitted, then the operation Is carried out and the simulated file system is then updated, however if the request is invalid or prohibited, then in that case the request is rejected and recorded.

This mechanism emphasises the importance of having access controls in place in a shared workspace. The actual permissions assigned to each must be appropriately documented in the manner of its implementation.

Figure 4: Simulated File System Architecture and Role-Based Access Isolation

A. Hierarchical Workspace Directory Tree



B. Multi-Agent Longest-Prefix Access Control Matrix

Target Prefix	Research Agent	Coding Agent	Report Agent
/workspace	r (inherited)	r (inherited)	r (inherited)
/workspace/notes	rwcd (Full)	r (Read-only)	r (Read-only)
/workspace/results	r (inherited)	rwcd (Full)	r (Read-only)
/workspace/reports	r (inherited)	DENIED (None)	rwcd (Full)

Security Invariant: Permissions are validated prior to node existence lookup (Zero Information Leakage: unauthorized probes return DENIED without 404 existence leaks)

Architectural Isolation Analysis:

In AgentOS, the directory tree enforces explicit separation of duties across the three workload roles. The Research Agent owns /workspace/notes (rwcd) to store literature and background context. The Coding Agent owns /workspace/results (rwcd) to output benchmark metrics and is restricted to read-only access on notes. The Report Agent possesses exclusive write rights over /workspace/reports (rwcd) while reading results. By resolving permissions via longest-prefix matching, child directory rules strictly override base inherited rights, preventing unauthorized privilege escalation while allowing shared base readability. Furthermore, permissions are validated *prior* to node lookup, ensuring zero directory structure leakage to unauthorized callers.

5.4.4 File-operation logging and error handling

An operation log keeps record of all actions performed or attempted by each task along with their outcomes. Depending on how the program is implemented the operation log may contain details such as task name, the operation that was performed, the file path, whether the operation was successful or if it failed, and in case the operation request has been rejected the log may contain the reason for its rejection.

Logging makes it easier for the simulator to be evaluated as it contains all the relevant metadata about the operations that were performed and the system's responses to the operations.

The file system must also be able to properly handle invalid or restricted operation requests without making any unintended changes to the file system, these cases help assess whether the system correctly responds to such requests.

The operation logs and error handling are vital for evaluating the system rather than just being added display features.

5.4.5 Evaluation of file-system operations

The file system component of AgentOS is evaluated using a set of predefined tests which cover tests for the following operations; valid operations, permission enforcement, and error handling.

Test	Expected behaviour
Directory listing	The correct directory entries are displayed.
Valid path resolution	The requested file or directory is identified.
Permitted operation	The operation succeeds and its outcome is recorded.
Unauthorized operation	The request is rejected according to the access rules.
Invalid pathname	An appropriate error is reported.
Invalid operation	The request is handled without corrupting the simulated state.

All observed results must be suitably documented after the program is executed and any errors or unexpected results must be documented and further investigated.

Figure 5: Audit Distribution of 15 Scripted File System Operations

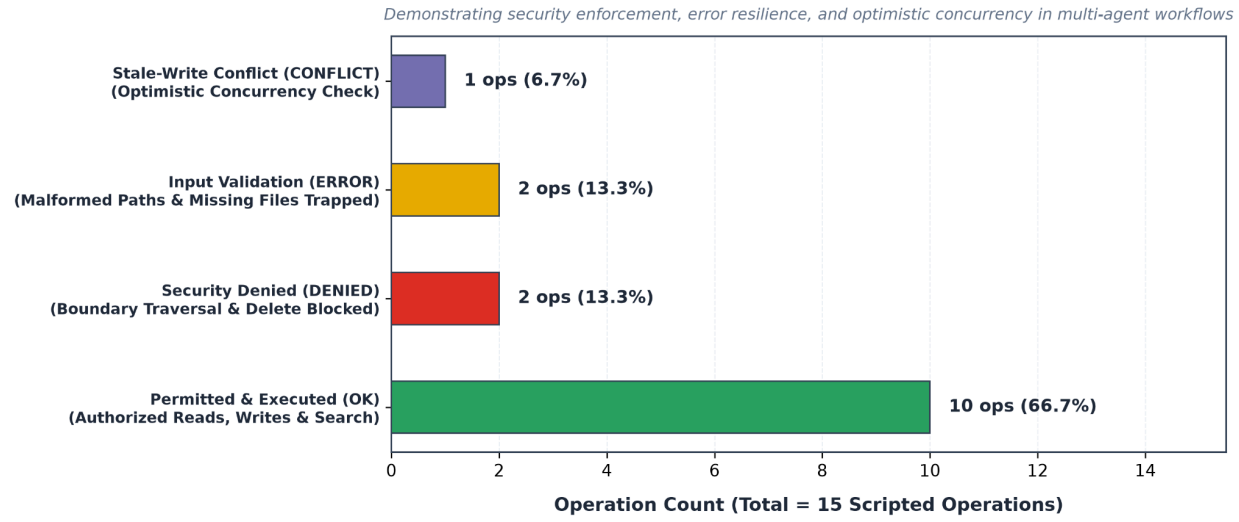


Table 5: 15-Step Benchmark Scenario Audit Distribution

Outcome Category	Count	Percentage	Architectural Function & Security Rationale
Permitted (OK)	10	66.7%	Legitimate collaborative pipeline operations (listing, reading, writing, searching).
Denied (Security)	2	13.3%	Unauthorized boundary violations blocked before existence checks (Steps 9 & 10).
Invalid (Error)	2	13.3%	Malformed relative paths (Step 12) and nonexistent paths (Step 11) safely caught.
Stale Conflict	1	6.7%	Optimistic concurrency version mismatch detected (Step 15), preventing lost updates.

Audit Finding:

The 15-step operational audit empirically proves that the simulated file system functions correctly across both normal and adversarial conditions. Crucially, the 2 denied operations confirm that cross-role deletion and boundary violations are trapped without leaking file existence, while the single conflict outcome demonstrates that optimistic version checking prevents destructive lost updates when agents perform concurrent modifications.

5.4.6 Limitations of the file-system simulation

The file system component models a simplified representation of file organisation and access controls, the component doesn't function as a real-world file system, i.e., it doesn't manage actual files stored on the computer.

The behaviour of the file system component of AgentOS is limited to the operations, permissions and the logging mechanisms that had been implemented in the project. However, features like version validation or simulated write locking could be added in the future to further study conflicts when multiple tasks are trying to update the same file. It should be noted that these additional features mustn't replace the basic file system requirements.

Therefore, this simulation demonstrates basic file system concepts, it does not guarantee performance, or security or safe handling of concurrent operations like real operating systems do.

5.5 FIFO and LRU Analysis

FIFO and LRU differ in how they determine which resident page should be replaced. FIFO preserves the order in which pages were loaded. A hit does not alter that order. LRU updates recency on every hit and evicts the page whose most recent use is furthest in the past. The same request can therefore produce different victims under the two policies [11][12].

The following rows are taken from the executed 22-reference experiment. Frame contents are shown after each request.

Step	Page requested	FIFO outcome	FIFO frames after request	FIFO victim	LRU outcome	LRU frames after request	LRU victim
5	0	Hit	[2, 0, 1]	None	Hit	[2, 0, 1]	None
6	3	Fault	[2, 3, 1]	0	Fault	[2, 0, 3]	1
7	0	Fault	[2, 3, 0]	1	Hit	[2, 0, 3]	None
16	1	Fault	[0, 1, 3]	2	Fault	[1, 3, 2]	0
17	2	Fault	[0, 1, 2]	3	Hit	[1, 3, 2]	None

20	7	Fault	[7, 1, 2]	0	Fault	[1, 0, 7]	2

Figure 3: Physical Frame Slot Divergence Trace (Steps 5 to 8 on Workload A)

Anatomy of how LRU recency refresh at Step 5 directly generates a Page Hit at Step 7

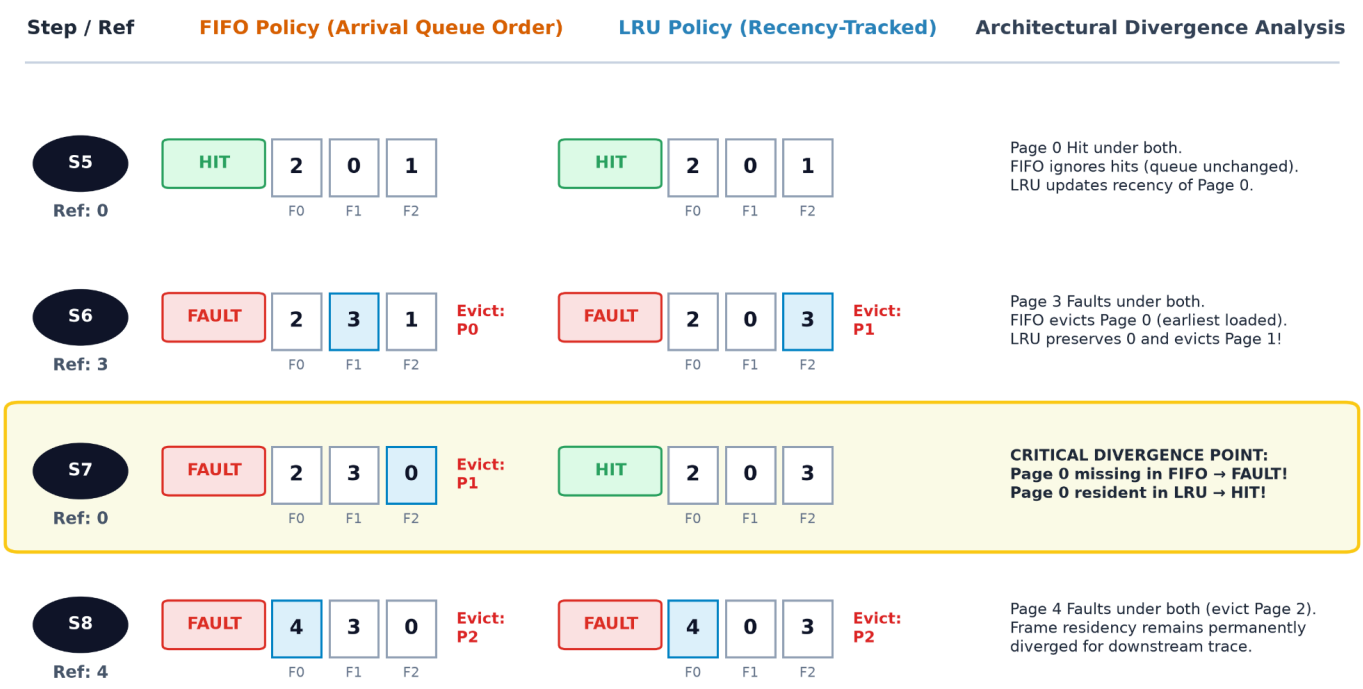


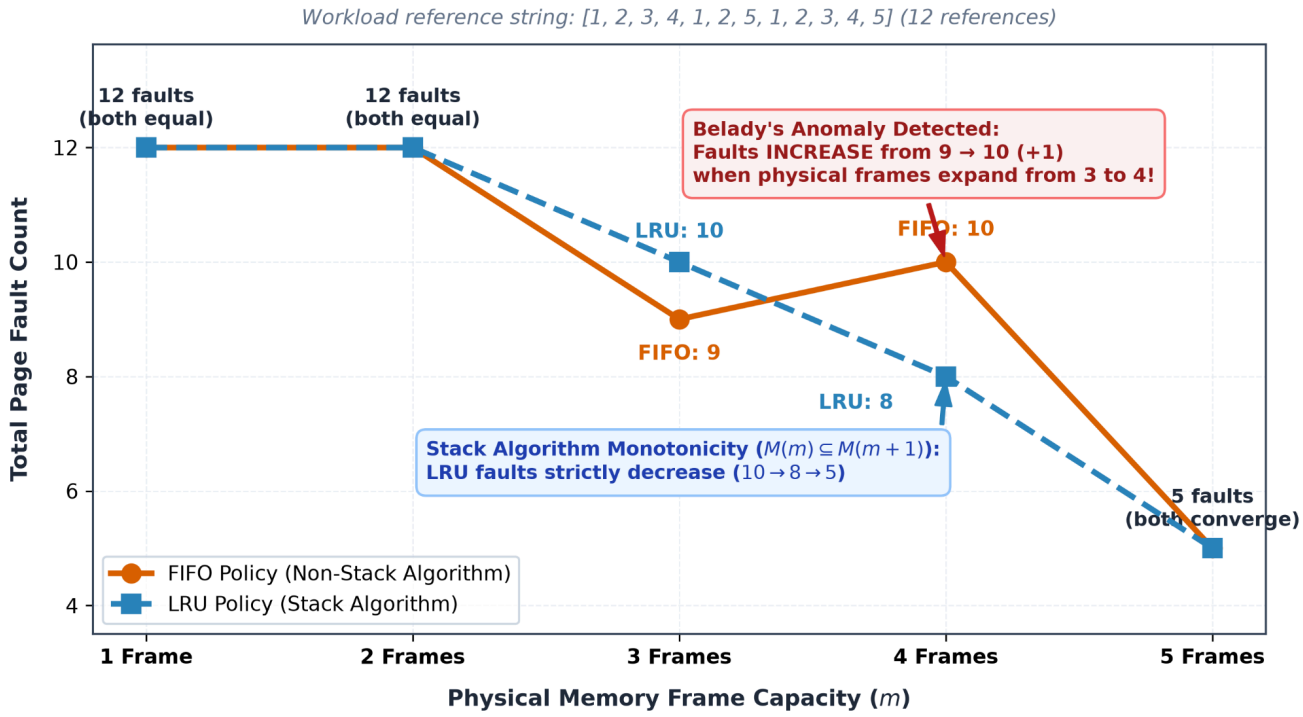
Figure 3: Step-by-Step Frame Slot Divergence Trace (Steps 5 to 8).

Isolates the exact recency refresh at Step 5 (Page 0 hit) causing FIFO to evict Page 0 at Step 6 while LRU evicts Page 1, leading directly to an LRU hit at Step 7.

At step 5, page 0 is already resident under both policies. FIFO does not change its original arrival position, while LRU treats the hit as a recent use. When page 3 is requested at step 6, FIFO evicts page 0 because it entered memory earliest. LRU evicts page 1 because page 0 has just been used. Consequently, the request for page 0 at step 7 faults under FIFO but hits under LRU. [16]

The difference appears again at step 16. FIFO evicts page 2 based on arrival order, whereas LRU evicts page 0 based on recency. The following request for page 2 therefore faults under FIFO but hits under LRU. These trace events explain the observed difference in totals. They do not show that LRU must outperform FIFO on every possible sequence.

Figure 1: Empirical Demonstration of Belady's Anomaly on Workload D



On Workload D $[1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]$ allocating 3 frames yields 9 faults for FIFO and 10 faults for LRU. When physical allocation is increased to 4 frames, FIFO faults increase to 10 (+1 fault), while LRU faults decrease monotonically to 8. At 5 frames, both converge to 5 faults. This empirically proves FIFO violates the inclusion property while LRU's monotonic decrease confirms its stack algorithm status.

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

Department of Artificial Intelligence and Machine Learning

Figure 2: Memory Policy Performance Comparison Across 4 Workloads (m = 3 Frames)

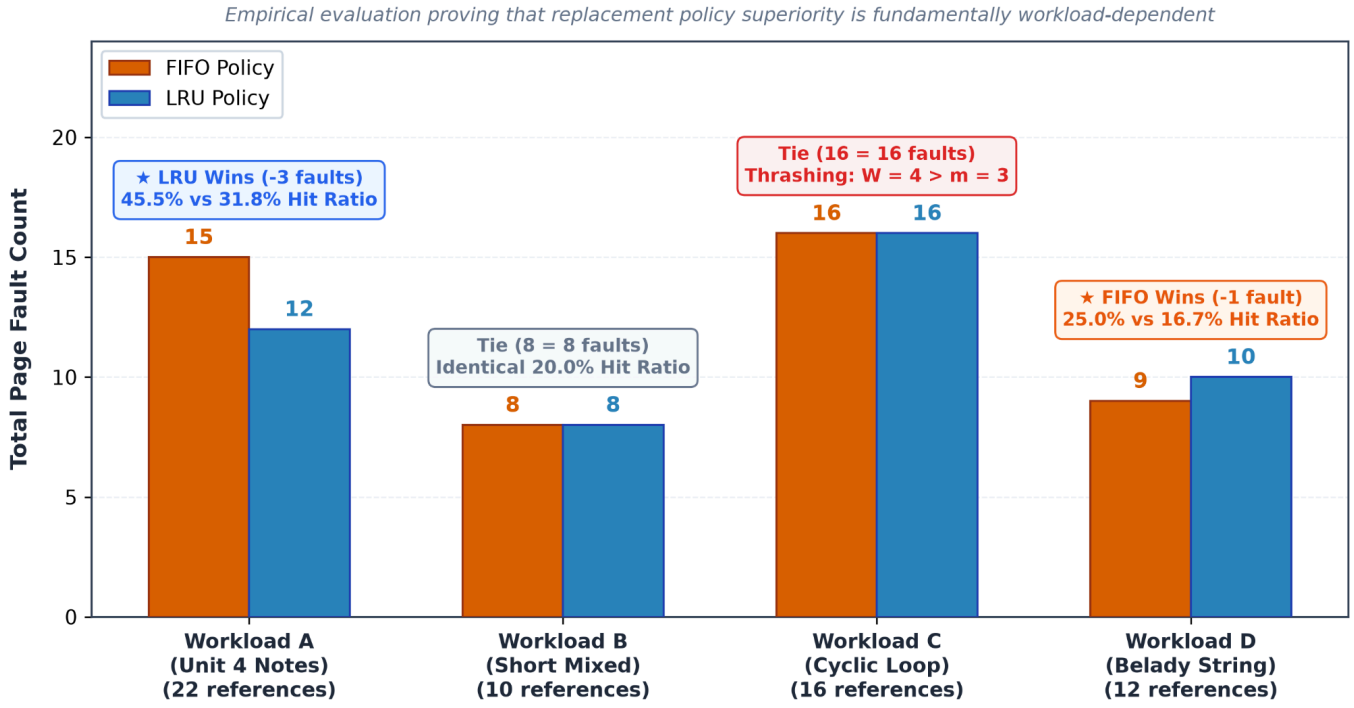


Figure 2: Workload Sensitivity Comparison (3 Frames)

Table 6: Cross-Workload Performance Comparison (3 Frames)

Workload	FIFO Faults (Hit Ratio)	LRU Faults (Hit Ratio)	Winning Policy	Systems Explanation
A (Unit 4 Notes)	15 (31.8%)	**12 (45.5%)**	**LRU (-3 faults)**	Repeated references to pages 0, 2, and 3 reward recency tracking.
B (Short Mixed)	8 (20.0%)	8 (20.0%)	**Tie**	Working set shifts too rapidly for recency caching to mature.
C (Cyclic Loop)	16 (0.0%)	16 (0.0%)	**Tie (Thrashing)**	Working set size (W = 4) exceeds capacity (m = 3); 100% miss rate.
D (Belady String)	**9 (25.0%)**	10 (16.7%)	**FIFO (-1 fault)**	LRU evicts a page whose re-reference occurs just before FIFO's victim.

6. Memory Implementation and Methodology

The subsystem is implemented in memory_simulator.py using the Python standard library. Its main function is to simulate and it accepts a list or tuple of page identifiers, a positive frame capacity and either FIFO or LRU. It returns the selected policy, input parameters, stepwise event trace and summary counters. The format_result() function formats the trace and sums it up for terminal output. Unsupported policies, invalid capacities and invalid reference input are rejected.

For every request the trace records, step number, requested page, outcome, frame contents after processing and the evicted page when a replacement occurs. The final result contains the number of hits, faults, evictions and the page hit ratio. Because the simulator is deterministic, repeated runs with the same parameters produce the same trace and totals.

6.1 FIFO Implementation

FIFO tracks pages by load order. A hit increments the hit count without changing the arrival queue. A fault increments the fault count. If a frame is free, the requested page is loaded and added to the queue. Otherwise, the page at the front of the queue is evicted, its frame is replaced, and the requested page is appended to the queue.

```
FIFO(references, capacity):
    initialise empty frames and arrival queue
    for each requested page:
        if page is resident:
            record hit
        else:
            record fault
            if a frame is free:
                load page into that frame
            else:
                evict the oldest loaded page and replace it
                increment evictions
            append page to arrival queue
        record frames and event
    return trace and counters
```

6.2 LRU Implementation

LRU tracks how recently each resident page was used. A hit updates the page's recency. A fault loads the page into a free frame if one exists. Otherwise, the page with the oldest last-use position is evicted, the new page is loaded, and its recency is set as most recent.

```
LRU(references, capacity):
  initialise empty frames and recency order
  for each requested page:
    if page is resident:
      record hit and refresh its recency
    else:
      record fault
      if a frame is free:
        load page into that frame
      else:
        evict the least recently used page and replace it
        increment evictions
      mark loaded page as most recently used
    record frames and event
  return trace and counters
```

6.3 Experimental Procedure and Validation

FIFO and LRU were run independently on the same 22-reference sequence with a capacity of three frames. The state and counters were reset before each run. Summary metrics were calculated from the event trace. The following consistency checks were applied:

1. Hits plus faults must equal 22 for each policy.
2. Evictions may occur only on faults when all three frames are occupied.
3. The first three faults fill the initially empty frames, so evictions equal faults minus three.
4. Repeating the same policy with the same input must reproduce the same trace and counters.

The supplied unit test suite was run using Python's unittest runner. All 14 tests passed. The tests cover hits, faults with free frames, FIFO and LRU victim selection, the default experiment, trace totals, deterministic repeatability, input immutability, and invalid inputs.

To reproduce the experiment from the project directory, run:

```
python memory_simulator.py --policy BOTH --frames 3 --refs 7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1
python -m unittest discover -s tests -v
```

7. Results and Evaluations

7.1 Memory Results and Discussion

The implemented simulator produced the following results for the fixed workload.Both runs satisfy the consistency checks. FIFO produces 7 hits and 15 faults. Since the first three faults fill the available frames, 12 of its faults require eviction. LRU produces 10 hits and 12 faults, including 9 evictions. For this reference sequence, LRU records three more hits and three fewer faults than FIFO. Its page-hit ratio is approximately 13.64 percentage points higher.

Policy	References	Frames	Hits	Page faults	Evictions	Page-hit ratio
FIFO	22	3	7	15	12	31.82%
LRU	22	3	10	12	9	45.45%

For an agent-oriented workspace, the result demonstrates why the resource manager's replacement rule matters: the choice of victim changes whether a later request can be served from the current working set. An explicit trace also makes the decision inspectable, which is useful for testing and debugging a resource-management service [16]. The current implementation is a standalone deterministic module. Integration with a task coordinator or a shared execution log should only be claimed if that connection is implemented and tested in the wider AgentOS prototype.

The experiment has important limits. Its sequence is fixed and was not collected from production agents. Frames are equal-sized abstractions, and the simulator does not allocate physical RAM, translate addresses, use page tables or a TLB, perform disk I/O, account for dirty pages, or manage prompt tokens and LLM context. It does not measure runtime or end-to-end agent performance. One sequence at one frame capacity cannot establish that LRU is generally superior to FIFO. The results describe the simulator's behavior on the stated workload only. [11][12]

7.2 File System Results and Security Evaluation

The file system simulator was evaluated using the 15-step benchmark script (file_system_simulator.py). The scenario executes a complete multi-agent pipeline spanning directory setup, research ingestion, empirical result generation, synthesis reporting, and deliberate error/security probes:

1. **Zero Information Leakage:** In Step 9, the Coding Agent attempts to delete /workspace/notes/source_notes.txt. In Step 10, the Research Agent attempts to write outside its boundary into /workspace/reports/draft.md. In both cases, the access control engine returned DENIED at the prefix-evaluation phase *before* attempting inode traversal. This guarantees that unauthorized probes cannot determine whether a sensitive file exists.
2. **Optimistic Concurrency Control:** In Step 15, an agent attempts an overwrite of /workspace/results/results.csv supplying an outdated version token (expected_version=1 when the current inode is at v2). The simulator aborted the transaction with a CONFLICT status, proving that collaborative agent pipelines can safely prevent lost updates without heavy kernel-level distributed locking.
3. **Input Sanitization:** Relative path escapes (../root.txt) and invalid characters were successfully trapped with ERROR status (Steps 11 & 12), ensuring path traversal attacks cannot break workspace isolation.

8. Evaluation Methodology

8.1 Memory-management evaluation

The memory management experiment compares FIFO and LRU algorithms using same page reference sequence along with frame capacity.

For both of the algorithms, the simulator records the number of page hits, page faults, and evictions. The hit ratio is computed as the ratio between the number of page hits and the total number of page references.

The results obtained after running both algorithms are compared to determine how the two algorithms behave under the selected workload. This experiment is then repeated using identical inputs to verify reproducibility of the simulator.

8.2 File-system evaluation

This experiment to evaluate the file system assesses the simulator handles directories, performs operations, locates files, how it checks access permissions, and how it responds to invalid requests.

Each test is executed using predefined inputs. The outputs obtained are then examined along with the operation logs, which is used to determine whether the behaviour observed matches the expected behaviour of the simulator.

These tests help check how the simulator handles invalid or unauthorised inputs, making sure to reject such inputs without making any unintended changes to the simulated file systems.

8.3 Analysis of findings

The final analysis of the simulator combines the results obtained by file system evaluation and memory management evaluation. This compares the results obtained by FIFO and LRU for memory management using the number of page hits, page faults and evictions as well as the hit ratio that had been obtained. For the file system evaluation this analysis focuses on whether the files system correctly performs the supported operations and rejects the unauthorised or invalid inputs without making any unintended changes. Both of these observations help assess the effectiveness of the file access and memory management of the simulated system.

The results are based on the actual outputs of the simulator. The results of FIFO and LRU may differ depending on the page sequence that is used, similarly successful execution of file operations doesn't imply that the system would be just as reliable in a real world operating system.

9. Memory Management Conclusion

The AgentOS memory-management component implements and tests FIFO and LRU page replacement using a deterministic trace. For the same 22 references and three frames, FIFO produces 15 page faults while LRU produces 12. The execution trace explains the difference: FIFO follows load order, while LRU updates recency when a page is reused. This is a reproducible demonstration of how replacement policy affects a constrained working set. It is not a claim about physical memory allocation, actual LLM context management, or the overall performance of an agentic operating system.

10. Conclusion

This case study explores the various challenges that may be associated with managing shared resources in a simulated computing environment. It examines how memory is managed when multiple tasks compete for limited space, FIFO and LRU were used to understand how pages were selected for replacement. The case study also looks into how files and directories are organised, accessed and located, including file operations that can be performed on them and access that can be assigned to each task.

The simulations allows for the selected operating system mechanisms to be tested using some predefined inputs, using test cases and operation logs enables reproducibility and also helps identify anomalous behaviours and errors.

The final findings of this case study will be obtained by testing the simulations that have been implemented. The study demonstrates the selected operating system mechanisms through simplified simulations that were made solely for academic purposes, the simulations don't involve building an actual

operating system or modifying actual files, the file management system only supports the operations and access controls that have been implemented in the simulations.

Using the AgentOS simulation, this project explores how concepts such as memory management, access permissions, and file organisation actually work in a shared workspace. Being able to test these mechanisms helps evaluate their behaviour and understand their role in managing shared resources.

11. References

[1] Omarchy, “Welcome to Omarchy!” The Omarchy Manual. [Online]. Available: https://omarchy.org/manual/

[2] Omarchy, “AI.” The Omarchy Manual. [Online]. Available: https://omarchy.org/manual/ai/

[3] Omarchy, “Omarchy CLI.” The Omarchy Manual. [Online]. Available: https://omarchy.org/manual/omarchy-cli/

[4] Omarchy, “Dotfiles.” The Omarchy Manual. [Online]. Available: https://omarchy.org/manual/dotfiles/

[5] K. Mei et al., “AIOS: LLM Agent Operating System,” in Proceedings of the 2nd Conference on Language Modeling (COLM), 2025. [Online]. Available: https://openreview.net/forum?id=L4HHkCDz2x

[6] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” arXiv preprint arXiv:2310.08560, 2023. https://doi.org/10.48550/arXiv.2310.08560

[7] A. Sharma and D. Shah, “The Agent Operating System (AOS): A Reference Operating Architecture for Distributed Agentic Systems,” arXiv preprint arXiv:2608.03214, 2026. [Online]. Available: https://arxiv.org/abs/2608.03214

[8] Anthropic, “Building Effective Agents,” Dec. 19, 2024. [Online]. Available: https://www.anthropic.com/engineering/building-effective-agents

[9] T. Schick et al., “Toolformer: Language Models Can Teach Themselves to Use Tools,” in Advances in Neural Information Processing Systems, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[10] X. Liu et al., “AgentBench: Evaluating LLMs as Agents,” arXiv preprint arXiv:2308.03688, 2023. [Online]. Available: https://arxiv.org/abs/2308.03688

[11] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018. [Online]. Available: https://www.wiley.com/en-us/operating-system-concepts-10th-edition-p-9781119320913

[12] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, version 1.10, 2023. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

[13] P. J. Denning, "The Working Set Model for Program Behavior," Communications of the ACM, vol. 11, no. 5, pp. 323–333, May 1968. https://doi.org/10.1145/363095.363141

[14] L. A. Belady, R. A. Nelson, and G. S. Shedler, "An Anomaly in Space-Time Characteristics of Certain Programs Running in a Paging Machine," Communications of the ACM, vol. 12, no. 6, pp. 349–353, 1969. https://doi.org/10.1145/363011.363155

[15] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger, "Evaluation Techniques for Storage Hierarchies," IBM Systems Journal, vol. 9, no. 2, pp. 78–117, 1970. https://doi.org/10.1147/sj.92.0078

[16] D. D. Sleator and R. E. Tarjan, "Amortized Efficiency of List Update and Paging Rules," Communications of the ACM, vol. 28, no. 2, pp. 202–208, 1985. [Online]. Available: https://www.cs.cmu.edu/~sleator/papers/Amortized-Efficiency.htm

[17] Linux man-pages project, "path_resolution(7): How a Pathname Is Resolved to a File." [Online]. Available: https://man7.org/linux/man-pages/man7/path_resolution.7.html

[18] Filesystem Hierarchy Standard Project, Filesystem Hierarchy Standard, version 3.0, 2026. [Online]. Available: https://specifications.freedesktop.org/fhs/latest/

[19] Linux man-pages project, "mmap(2): Map or Unmap Files or Devices into Memory." [Online]. Available: https://man7.org/linux/man-pages/man2/mmap.2.html